

Department of Computer Science and Engineering
III/II CSE - R23 Regulations
BIG DATA ANALYTICS

UNIT II - NO SQL DATA MANAGEMENT

Working with Cassandra, Table creation, loading and reading data.

Working with Cassandra – Table Creation, Loading and Reading Data

Apache Cassandra is a distributed NoSQL database designed for handling large amounts of data across many servers without a single point of failure. Cassandra uses **CQL (Cassandra Query Language)** which is similar to SQL for performing database operations such as creating tables, inserting data, and retrieving records.

Procedure

Step 1: Create Keyspace

```
CREATE KEYSPACE college  
WITH replication = {  
'class' : 'SimpleStrategy',  
'replication_factor' : 3  
};
```

Step 2: Use Keyspace

```
USE college;
```

Step 3: Create Table

```
CREATE TABLE students (  
student_id INT PRIMARY KEY,  
name TEXT,  
department TEXT,  
age INT  
);
```

Step 4: Insert Data (Loading Data)

```
INSERT INTO students (student_id, name, department, age)  
VALUES (1, 'Arun', 'CSE', 21);
```

Step 5: Insert Another Record

```
INSERT INTO students (student_id, name, department, age)  
VALUES (2, 'Meena', 'ECE', 22);
```

Step 6: Read Data

```
SELECT * FROM students;
```

Step 7: Read Specific Columns

```
SELECT name, department FROM students;
```

Step 8: Read Data with Condition

```
SELECT * FROM students  
WHERE student_id = 1;
```

Step 9: Update Data

```
UPDATE students  
SET age = 23  
WHERE student_id = 2;
```

Step 10: Delete Data

```
DELETE FROM students  
WHERE student_id = 1;
```

The query:

```
CREATE KEYSPACE college  
WITH replication = {  
'class' : 'SimpleStrategy',  
'replication_factor' : 3  
};
```

is used in **Apache Cassandra** to **create a keyspace** (similar to a database in relational systems).

Let's break it down step-by-step.

1. CREATE KEYSPACE college

This command creates a **keyspace named college**.

In Cassandra:

Keyspace = Database (in MySQL / SQL Server)

It stores **tables and data**.

Example structure:

Keyspace: college

- └── students table
- └── faculty table
- └── courses table

2. WITH replication

This defines **how data is replicated across nodes in the Cassandra cluster**.

Cassandra distributes data across multiple machines to ensure:

High availability
Fault tolerance
No single point of failure

3. 'class' : 'SimpleStrategy'

This defines the **replication strategy**.

SimpleStrategy means:

Used for **single data center clusters**

Data is replicated sequentially to nodes.

Other strategies exist, such as:

NetworkTopologyStrategy → used for **multiple data centers**.

Example:

Node1 → Node2 → Node3

4. 'replication_factor' : 3

This defines **how many copies of the data Cassandra will store**.

replication_factor = 3 means:

Each piece of data will be stored **3 times on different nodes**.

Example:

Node Data

Node1 Copy 1

Node2 Copy 2

Node3 Copy 3

Benefits:

If **one node fails**, data is still available.

Improves **fault tolerance and reliability**.

5. After Creating Keyspace

You must select the keyspace before creating tables:

USE college;

Then create tables like:

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name TEXT,  
  department TEXT  
);
```

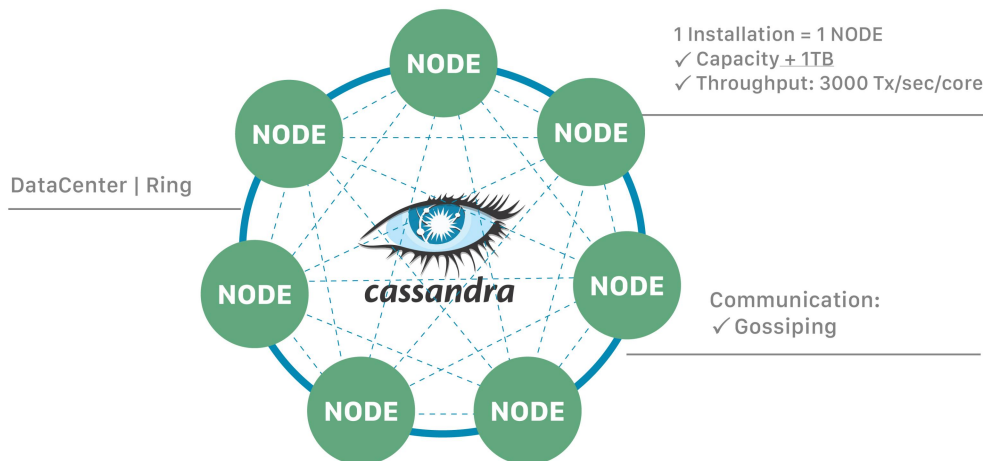
In simple terms

This query means:

Create a **database named college** and store **3 copies of each data record across different servers** for reliability.

Apache Cassandra Architecture (Simple Explanation)

ApacheCassandra™ = NoSQL Distributed Database



1. Cluster

A **Cluster** is the **top-level component** in Cassandra.

It contains **multiple nodes (servers)**.

All nodes work together to store and manage data.

There is **no master node**, all nodes are equal (peer-to-peer architecture).

Example:

Cluster

- Node 1
- Node 2
- Node 3
- Node 4

2. Node

A **Node** is a **single machine/server** in the Cassandra cluster.

Each node:

Stores a **portion of data**

Handles **read and write requests**

Replicates data to other nodes

Example:

Node 1 → stores part of student data

Node 2 → stores another part

Node 3 → stores replicated copies

3. Keyspace

A **Keyspace** is similar to a **database in SQL systems**.

It defines:

Tables

Replication strategy

Replication factor

Example:

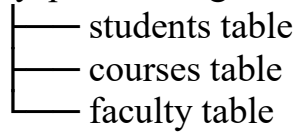
```
CREATE KEYSPACE college
```

```
WITH replication = {  
'class': 'SimpleStrategy',
```

```
'replication_factor':3  
};
```

Keyspace structure:

Keyspace: college



4. Table (Column Family)

A **Table** in Cassandra is similar to a **table in relational databases**, but internally it is called a **Column Family**.

Example:

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name TEXT,  
  department TEXT  
);
```

Example data:

student_id	name	department
1	Arun	CSE
2	Meena	ECE

5. Row

A **Row** represents a single record.

Example:

```
student_id = 1  
name = Arun  
department = CSE
```

6. Column

A **Column** stores a single piece of information.

Example:

```
student_id  
name  
department
```

Cassandra Hierarchy (Important for Exams)

Cluster



Node



Keyspace



Table (Column Family)



Row

↓
Column

Why Cassandra is Used in Big Data?

Advantages:

Highly scalable

Distributed storage

No single point of failure

High write performance

Used by companies like **Netflix**, **Apple**, and **Facebook**.